

231012133

```
private class MenuItemOpenClickListener implements ActionListener {

    //Write code here

    @Override

    public void actionPerformed(ActionEvent e) {

        // Display JFileChooser open dialog

        JFileChooser fileChooser = new JFileChooser();

        int result = fileChooser.showOpenDialog(Mini3GUI.this);

        if (result == fileChooser.APPROVE_OPTION) {

            File selectedFile = fileChooser.getSelectedFile();

            try {

                // Clear previous content to avoid duplicates

                taContent.setText("");

                cdp.clearRecords();

                // Read the file using BufferedReader

                BufferedReader reader = new BufferedReader(new FileReader(selectedFile));

                String line;

                int recordCount = 0;

                StringBuilder displayText = new StringBuilder();

                // Add header

                displayText.append("Date\t\tStart Time\tEnd\nTime\tDuration\tNumber\t\tProvider\n");

                displayText.append("=".repeat(100)).append("\n");
```

```

while ((line = reader.readLine()) != null) {
    line = line.trim();
    // Skip empty lines
    if (line.isEmpty()) {
        continue; }
    try {
        // convert fields into DetailRecord
        DetailRecord record = new DetailRecord(line);
        // Add CallDetailProcessor.addDR() to add the record to the collection
        cdp.addDR(record);
        recordCount++;
        // Display the record in text area with tab characters and new lines
        displayText.append(record.getDate()).append("\t")
            .append(record.getStartTime()).append("\t")
            .append(record.getEndTime()).append("\t")
            .append(record.getDuration()).append("\t")
            .append(record.getNumber()).append("\t")
            .append(record.getProvider()).append("\n");
    }
    catch (NumberFormatException | ArrayIndexOutOfBoundsException ex) {
        // Handle malformed records and invalid numeric fields safely
        System.out.println("Malformed record skipped: " + line); }
}
reader.close();

```

```

// Display the records in the text area

taContent.setText(displayText.toString());

// Update file

String fileName = selectedFile.getName();

lblFileStatus.setText("File loaded successfully: " + recordCount + " rows");

lblFileStatus.setForeground(Color.blue);

}

catch (IOException ioException) {

    lblFileStatus.setText("Error reading file: " + ioException.getMessage());

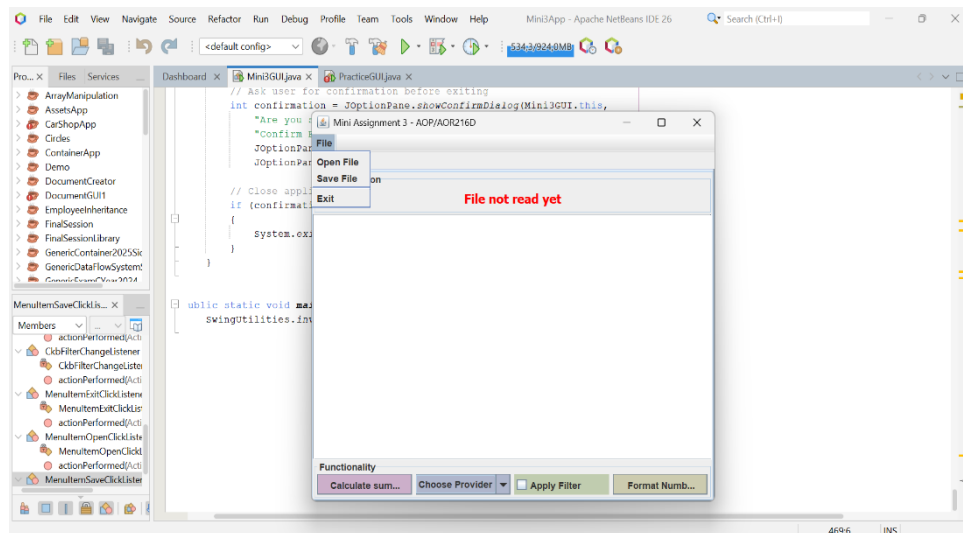
    lblFileStatus.setForeground(Color.red);

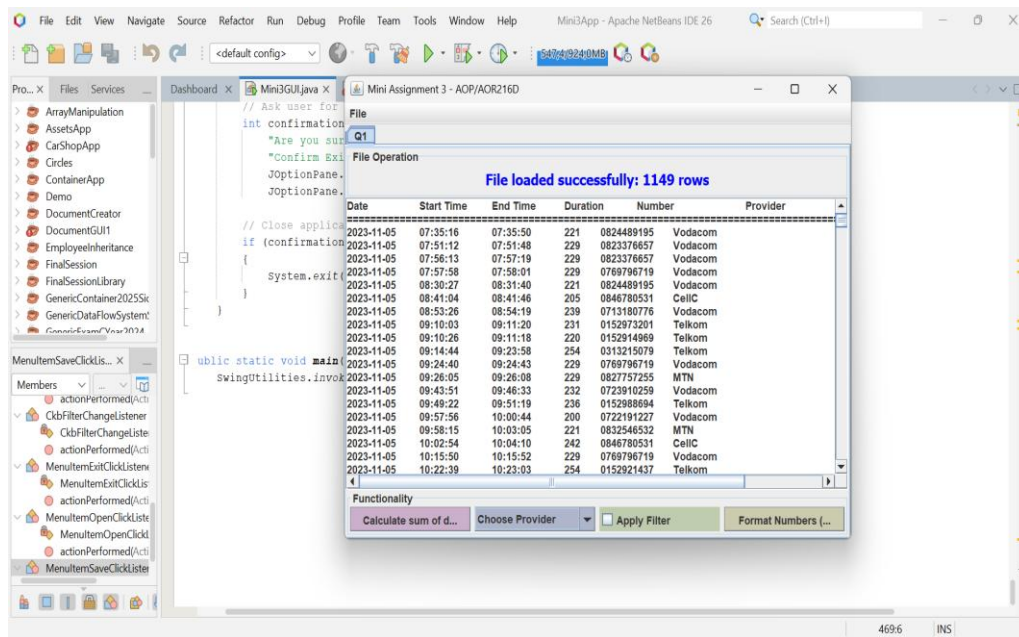
    ioException.printStackTrace(); }

} }

}

```





private class BtnCalculateDurationsClickListener implements ActionListener {

 //Write code here

 @Override

 public void actionPerformed(ActionEvent e) {

 ArrayList<DetailRecord> records = cdp.getRecords();

 // Handle the case where no file has been loaded or no records exist

 if (records == null || records.isEmpty()) {

 JOptionPane.showMessageDialog(Mini3GUI.this,

 "No records loaded. Please open a file first.",

 "No Data",

 JOptionPane.WARNING_MESSAGE);

 return;

 }

```
// Calculate the sum of all call durations
```

```
long totalDuration = 0;
```

```
for (int i = 0; i < records.size(); i++) {
```

```
    DetailRecord record = records.get(i);
```

```
    totalDuration += record.getDuration(); }
```

```
// Display the total using JOptionPane
```

```
JOptionPane.showMessageDialog(Mini3GUI.this,
```

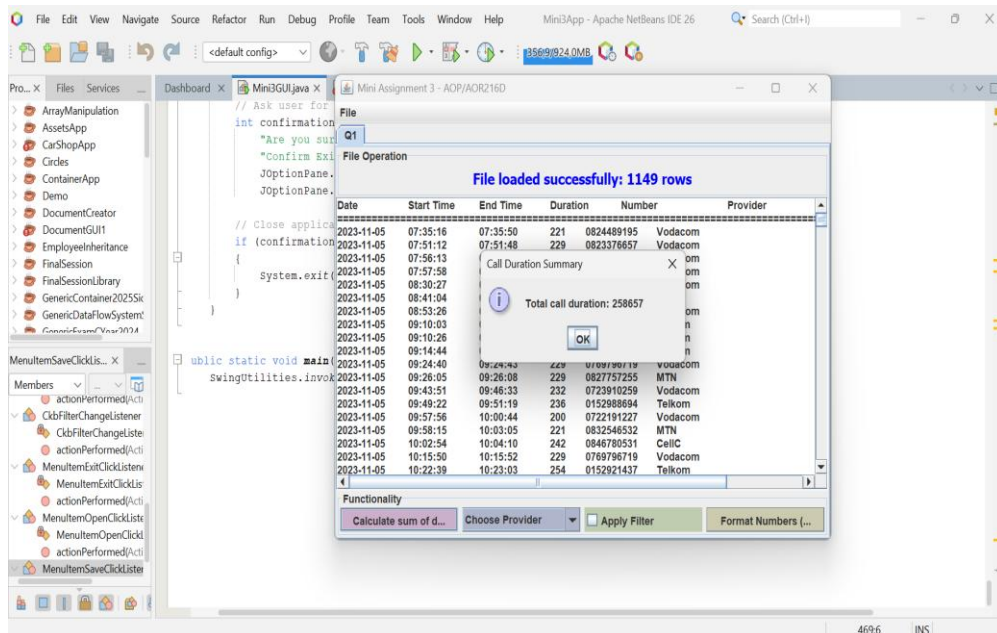
```
    "Total call duration: " + totalDuration,
```

```
    "Call Duration Summary",
```

```
    JOptionPane.INFORMATION_MESSAGE);
```

```
}
```

```
}
```



```

private class CkbFilterChangeListener implements ActionListener {

    //Write code here

    @Override

    public void actionPerformed(ActionEvent e) {

        // Get all collection

        ArrayList<DetailRecord> allRecords = cdp.getRecords();

        // Validate exist

        if (allRecords == null || allRecords.isEmpty()) {

            JOptionPane.showMessageDialog(Mini3GUI.this,

                "No records loaded. Please open a file first.",

                "No Data",

                JOptionPane.WARNING_MESSAGE);

            ckbFilter.setSelected(false);

            return; }

        StringBuilder displayText = new StringBuilder();

        // Check filter

        if (ckbFilter.isSelected()) {

            // Read the combo box

            String selectedProvider = (String) cmbProviders.getSelectedItem();

            // Validate selection

            if (selectedProvider.equals("Choose Provider")) {

                JOptionPane.showMessageDialog(Mini3GUI.this,

                    "Please select a provider to filter.",

                    "Select Provider",

                    JOptionPane.WARNING_MESSAGE);

```

```

ckbFilter.setSelected(false);

    return;
}

// Add header with same formatting as complete dataset
displayText.append("Date\t\tStart Time\tEnd
Time\tDuration\tNumber\t\tProvider\n");

displayText.append("=".repeat(100)).append("\n");

// Filter and display only records matching the selected provider
int matchCount = 0;
for (int i = 0; i < allRecords.size(); i++) {
    DetailRecord record = allRecords.get(i);
    if (record.getProvider().equalsIgnoreCase(selectedProvider)) {
        displayText.append(record.getDate()).append("\t")
            .append(record.getStartTime()).append("\t")
            .append(record.getEndTime()).append("\t")
            .append(record.getDuration()).append("\t")
            .append(record.getNumber()).append("\t")
            .append(record.getProvider()).append("\n");
        matchCount++;
    }
}
}

```

```

        // If no records match, show a clear message
        if (matchCount == 0) {
            displayText = new StringBuilder();
            displayText.append("No records found for
provider:").append(selectedProvider);
        }
    } else
    {
        // When filter is cleared, display all records again
        displayText.append("Date\t\tStart Time\tEnd
Time\tDuration\tNumber\t\tProvider\n");
        displayText.append("=".repeat(100)).append("\n");

        for (int i = 0; i < allRecords.size(); i++) {
            DetailRecord record = allRecords.get(i);
            displayText.append(record.getDate()).append("\t")
                .append(record.getStartTime()).append("\t")
                .append(record.getEndTime()).append("\t")
                .append(record.getDuration()).append("\t")
                .append(record.getNumber()).append("\t")
                .append(record.getProvider()).append("\n");
        }
    }

    // Update the text area display
    taContent.setText(displayText.toString()); }
}

```

MiniApp - Apache NetBeans IDE 26

File loaded successfully: 1149 rows

Date	Start Time	End Time	Duration	Number	Provider
2023-11-05	07:35:16	07:35:50	221	0824489195	Vodacom
2023-11-05	07:51:12	07:51:48	229	0823376657	Vodacom
2023-11-05	07:56:13	07:57:19	229	0823376657	Vodacom
2023-11-05	07:57:58	07:58:01	229	0769796719	Vodacom
2023-11-05	08:30:27	08:31:40	221	0824489195	Vodacom
2023-11-05	08:41:04	08:41:46	205	0846780531	CellC
2023-11-05	08:53:28	08:54:19	239	0713180776	Vodacom
2023-11-05	09:10:03	09:11:20	231	0152973201	Telkom
2023-11-05	09:10:26	09:11:18	220	0152914969	Telkom
2023-11-05	09:14:44	09:23:58	254	0313215079	Telkom
2023-11-05	09:24:40	09:24:43	229	0769796719	Vodacom
2023-11-05	09:26:05	09:26:08	229	0827757255	MTN
2023-11-05	09:43:51	09:43:51	239	0723910259	Vodacom
2023-11-05	09:49:22	09:49:22	239	0152988694	Telkom
2023-11-05	09:57:56	09:57:56	229	0722191227	Vodacom
2023-11-05	09:58:15	09:58:15	229	0832546532	MTN
2023-11-05	10:02:54	10:02:54	242	0846780531	CellC
2023-11-05	10:15:50	10:15:50	229	0769796719	Vodacom
2023-11-05	10:22:39	10:22:39	254	0152921437	Telkom

Functionality: Calculate sum of d... Choose Provider: Vodacom Apply Filter Format Numbers [...]

MiniApp - Apache NetBeans IDE 26

File loaded successfully: 1149 rows

Date	Start Time	End Time	Duration	Number	Provider
2023-11-05	07:35:16	07:35:50	221	0824489195	Vodacom
2023-11-05	07:51:12	07:51:48	229	0823376657	Vodacom
2023-11-05	07:56:13	07:57:19	229	0823376657	Vodacom
2023-11-05	07:57:58	07:58:01	229	0769796719	Vodacom
2023-11-05	08:30:27	08:31:40	221	0824489195	Vodacom
2023-11-05	08:41:04	08:41:46	205	0846780531	CellC
2023-11-05	08:53:28	08:54:19	239	0713180776	Vodacom
2023-11-05	09:10:03	09:11:20	231	0152973201	Telkom
2023-11-05	09:10:26	09:11:18	220	0152914969	Telkom
2023-11-05	09:14:44	09:23:58	254	0313215079	Telkom
2023-11-05	09:24:40	09:24:43	229	0769796719	Vodacom
2023-11-05	09:26:05	09:26:08	229	0827757255	MTN
2023-11-05	09:43:51	09:46:33	232	0723910259	Vodacom
2023-11-05	09:49:22	09:51:19	236	0152988694	Telkom
2023-11-05	09:57:56	10:00:44	260	0722191227	Vodacom
2023-11-05	09:58:15	10:03:05	221	0832546532	MTN
2023-11-05	10:02:54	10:04:10	242	0846780531	CellC
2023-11-05	10:15:50	10:15:52	229	0769796719	Vodacom
2023-11-05	10:22:39	10:23:03	254	0152921437	Telkom

Functionality: Calculate sum of d... MTN Apply Filter Format Numbers [...]

MiniApp - Apache NetBeans IDE 26

File loaded successfully: 1149 rows

Date	Start Time	End Time	Duration	Number	Provider
2023-11-23	19:19:02	19:21:26	200	0782911731	MTN
2023-11-23	20:46:36	20:49:37	239	0732730486	MTN
2023-11-23	20:54:04	20:56:46	200	0833634717	MTN
2023-11-23	20:58:54	20:59:29	200	0782911731	MTN
2023-11-24	07:50:09	07:51:40	239	0784777790	MTN
2023-11-24	08:04:26	08:07:07	206	0732730486	MTN
2023-11-24	08:52:05	08:52:10	231	0731749636	MTN
2023-11-24	08:52:47	08:52:49	231	0731749636	MTN
2023-11-24	11:11:59	11:13:45	205	0784777790	MTN
2023-11-24	13:23:11	13:23:14	200	0787318103	MTN
2023-11-24	13:23:44	13:24:46	200	0787318103	MTN
2023-11-24	15:22:41	15:23:17	200	0738293892	MTN
2023-11-24	15:29:21	15:30:10	234	0784777790	MTN
2023-11-24	15:47:51	15:47:58	200	0787318103	MTN
2023-11-24	15:48:25	15:48:29	200	0787318103	MTN
2023-11-24	16:19:39	16:19:49	200	0787318103	MTN
2023-11-24	16:21:29	16:22:31	200	0787318103	MTN
2023-11-24	16:25:37	16:25:43	200	0731799602	MTN
2023-11-30	11:29:29	11:29:51	234	0733142945	MTN
2023-11-30	17:56:25	17:57:20	233	0836701733	MTN

Functionality: Calculate sum of d... MTN Apply Filter Format Numbers [...]

```

private class BtnFormatNumbersClickListener implements ActionListener {

    @Override

    public void actionPerformed(ActionEvent e) {

        ArrayList<DetailRecord> records = cdp.getRecords();

        // Handle the case where no file has been loaded or no records exist
        if (records == null || records.isEmpty()) {

            JOptionPane.showMessageDialog(Mini3GUI.this,

                "No records loaded. Please open a file first.",

                "No Data",

                JOptionPane.WARNING_MESSAGE);

            return;

        }

        // Update all phone numbers to include +27 prefix
        for (int i = 0; i < records.size(); i++) {

            DetailRecord record = records.get(i);

            String phoneNumber = record.getPhoneNumber();

            // Check if the number starts with 0 (South African local format)
            if (phoneNumber.startsWith("0")) {

                // Remove leading zero and prefix +27

                String formattedNumber = "+27" + phoneNumber.substring(1);

                record.setPhoneNumber(formattedNumber); }

        }

        StringBuilder displayText = new StringBuilder();

        displayText.append("Date\t\tStart Time\tEnd
Time\tDuration\tNumber\t\tProvider\n");

        displayText.append("=".repeat(100)).append("\n");

```

```

for (int i = 0; i < records.size(); i++) {

    DetailRecord record = records.get(i);

    displayText.append(record.getDate()).append("\t")

        .append(record.getStartTime()).append("\t")

        .append(record.getEndTime()).append("\t")

        .append(record.getDuration()).append("\t")

        .append(record.getNumber()).append("\t")

        .append(record.getProvider()).append("\n");

}

// Update the text area with formatted numbers

taContent.setText(displayText.toString());

// Show confirmation message

JOptionPane.showMessageDialog(Mini3GUI.this,

    "All phone numbers have been formatted with +27 prefix.",

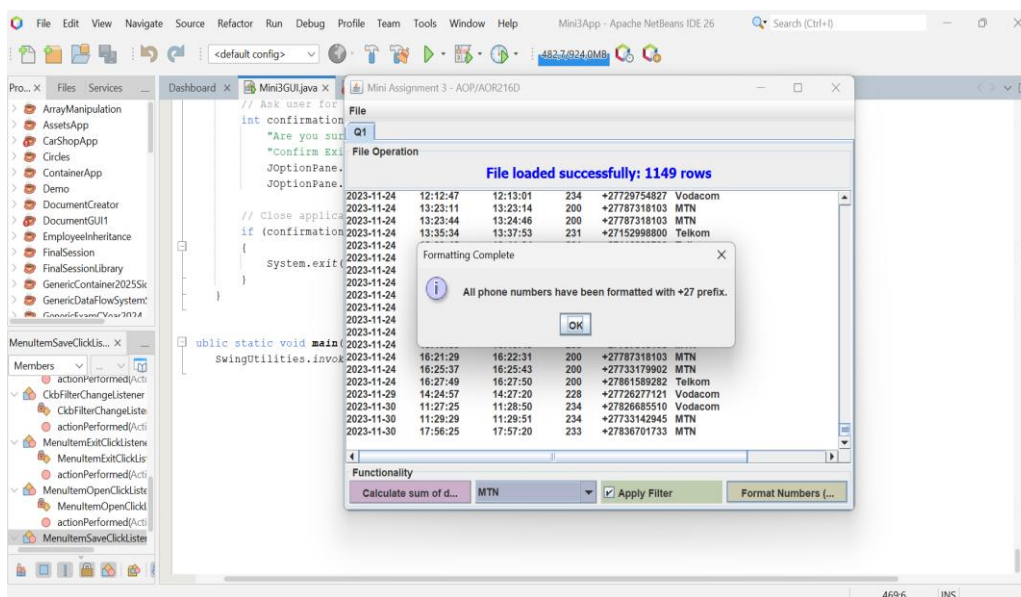
    "Formatting Complete",

    JOptionPane.INFORMATION_MESSAGE);

}

}

```



```

private class MenuItemSaveClickListener implements ActionListener {

    //Write code here

    @Override

    public void actionPerformed(ActionEvent e) {

        // Question 5 - Save processed records

        ArrayList<DetailRecord> records = cdp.getRecords();

        // Check if records exist

        if (records == null || records.isEmpty()) {

            JOptionPane.showMessageDialog(Mini3GUI.this,

                "No records to save. Please load a file first.",

                "No Data",

                JOptionPane.WARNING_MESSAGE);

            return;

        }


        // Open JFileChooser save dialog

        JFileChooser fileChooser = new JFileChooser();

        int result = fileChooser.showSaveDialog(Mini3GUI.this);

        // Continue only when user approves save destination

        if (result == JFileChooser.APPROVE_OPTION) {

            File selectedFile = fileChooser.getSelectedFile();

            try {

                // Write processed records to selected file using PrintWriter and FileWriter

                PrintWriter writer = new PrintWriter(new FileWriter(selectedFile));

```

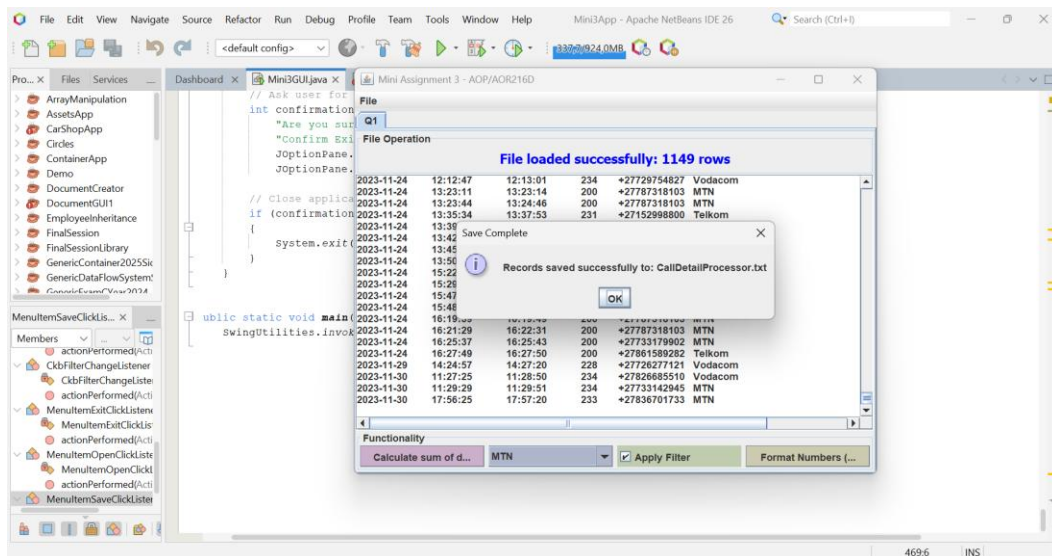
```

// Write records in required output format
for (int i = 0; i < records.size(); i++) {
    DetailRecord record = records.get(i);
    writer.println(record.getDate() + ";" +
        record.getStartTime() + ";" +
        record.getEndTime() + ";" +
        record.getDuration() + ";" +
        record.getPhoneNumber() + ";" +
        record.getProvider());
}
writer.close();

// Display message after successful saving
JOptionPane.showMessageDialog(Mini3GUI.this,
    "Records saved successfully to: " + selectedFile.getName(),
    "Save Complete",
    JOptionPane.INFORMATION_MESSAGE);
}

catch (IOException ex) {
    JOptionPane.showMessageDialog(Mini3GUI.this,
        "Error saving file: " + ex.getMessage(),
        "Save Error",
        JOptionPane.ERROR_MESSAGE); }
}
}
}

```



```
private class MenuItemExitClickListener implements ActionListener {

    @Override

    public void actionPerformed(ActionEvent e) {

        // Question 6 - Exit through menu

        // Ask user for confirmation before exiting

        int confirmation = JOptionPane.showConfirmDialog(Mini3GUI.this,

            "Are you sure you want to exit?", "Confirm Exit",

            JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);

        // Close application only when user approves

        if (confirmation == JOptionPane.YES_OPTION) {

            System.exit(0); }

    }

}
```

